

Working with Data in Python Cheat Sheet

Reading and writing files

Package/Method	Description	Syntax and Code Example
File opening modes	Different modes to open files for specific operations.	Syntax: <code>r</code> (reading) <code>w</code> (writing) <code>a</code> (appending) <code>b</code> (binary, otherwise text) Example: <code>with open('data.txt', 'r') as file: content = file.read() print(content)</code> <code>with open('output.txt', 'w') as file: file.write('Hello, world!')</code> <code>with open('log.txt', 'a') as file: file.write('Log entry: Something happened.')</code> <code>with open('data.txt', 'rb') as file: content = file.read() file.write('Updated content: ' + content + '\n')</code>
File reading methods	Different methods to read file content in various ways.	Syntax: <code>file.readlines()</code> # Reads all lines as a list <code>readlines()</code> # Reads the next line as a string <code>file.read()</code> # Reads the entire file content as a string Example: <code>with open('data.txt', 'r') as file: lines = file.readlines() next_line = file.readline() content = file.read()</code>
File writing methods	Different write methods to write content to a file.	Syntax: <code>file.write(content)</code> # Writes a string to the file <code>file.writelines()</code> # Writes a list of strings to the file Example: <code>lines = ['Hello!', 'World!'] with open('output.txt', 'w') as file: file.writelines(lines)</code>
Iterating over lines	Iterates through each line in the file using a "loop".	Syntax: <code>for line in file:</code> # Code to process each line Example: <code>with open('data.txt', 'r') as file: for line in file: print(line)</code>
Open() and close()	Opens a file, performs operations, and explicitly closes the file using the <code>close()</code> method.	Syntax: <code>file = open(filename, mode)</code> # Code that uses the file <code>file.close()</code> Example: <code>file = open('data.txt', 'r') content = file.read() file.close()</code>
with open()	Opens a file using a <code>with</code> block, ensuring automatic file closure after usage.	Syntax: <code>with open(filename, mode) as file:</code> # Code that uses the file Example: <code>with open('data.txt', 'r') as file: content = file.read()</code>

Pandas		
Package/Method	Description	Syntax and Code Example
<code>read_csv()</code>	Reads data from a ".CSV" file and creates a DataFrame.	Syntax: <code>dataframe_name = pd.read_csv('filenames.csv')</code> Example: <code>df = pd.read_csv('data.csv')</code>
<code>read_excel()</code>	Reads data from an Excel file and creates a DataFrame.	Syntax: <code>dataframe_name = pd.read_excel('filename.xlsx')</code> Example: <code>df = pd.read_excel('data.xlsx')</code>
<code>to_csv()</code>	Writes DataFrame to a CSV file.	Syntax: <code>dataframe_name.to_csv('output.csv', index=False)</code> Example: <code>df.to_csv('output.csv', index=False)</code>
Access Columns	Accesses a specific column using <code>[]</code> in the DataFrame.	Syntax: <code>dataframe_name['column_name']</code> # Accesses single column <code>dataframe_name[['column1', 'column2']]</code> # Accesses multiple columns Example: <code>df['age'] df[['name', 'age']]</code>
<code>describe()</code>	Generates statistics summary of numeric columns in the DataFrame.	Syntax: <code>dataframe_name.describe()</code> Example: <code>df.describe()</code>
<code>drop()</code>	Removes specified rows or columns from the DataFrame. <code>axis=1</code> indicates columns. <code>axis=0</code> indicates rows.	Syntax: <code>dataframe_name.drop(['column1', 'column2'], axis=1, inplace=True)</code> <code>dataframe_name.drop(index=[row1, row2], axis=0, inplace=True)</code> Example: <code>df.drop('age', 'salary', axis=1, inplace=True)</code> # Will drop columns <code>df.drop(index=[5, 10], axis=0, inplace=True)</code> # Will drop rows

dropna()	Removes rows with missing NaN values from the DataFrame. axis=0 indicates rows.	Syntax: <pre>dataframe.dropna(axis=0, inplace=True)</pre> Example: <pre>df.dropna(axis=0, inplace=True)</pre>
duplicated()	Duplicate or repetitive values or records within a data set.	Syntax: <pre>dataframe.duplicated()</pre> Example: <pre>duplicates_row = df[df.duplicated()]</pre>
Filter Rows	Creates a new DataFrame with rows that meet specified conditions.	Syntax: <pre>filtered_df = dataframe[(Conditional_Statements)]</pre> Example: <pre>filtered_df = df[(df["age"] > 30) & (df["salary"] < 50000)]</pre>
groupby()	Splits a DataFrame into groups based on specified criteria, enabling subsequent aggregation, transformation, or analysis within each group.	Syntax: <pre>grouped = dataframe.groupby(by = column, level=None, as_index=True, sort=True, group_keys=True, observed=False, dropna=False, engine=None)</pre> Example: <pre>grouped = df.groupby(["category", "region"]).agg({"sales": "sum"})</pre>
head()	Displays the first n rows of the DataFrame.	Syntax: <pre>dataframe.head(n)</pre> Example: <pre>df.head()</pre>
Import pandas	Imports the Pandas library with the alias pd.	Syntax: <pre>import pandas as pd</pre> Example: <pre>import pandas as pd</pre>
info()	Provides information about the DataFrame, including data types and memory usage.	Syntax: <pre>dataframe.info(verbose)</pre> Example: <pre>df.info()</pre>
merge()	Merges two DataFrames based on multiple common columns.	Syntax: <pre>merged_df = pd.merge(df1, df2, on=["column1", "column2"])</pre> Example: <pre>merged_df = pd.merge(sales, products, on=["product_id", "category_id"])</pre>
print DataFrame	Displays the content of the DataFrame.	Syntax: <pre>print(df) # or just type df</pre> Example: <pre>print(df) df</pre>
replace()	Replaces specific values in a column with new values.	Syntax: <pre>dataframe.replace("column_name", replace_old_value, new_value, inplace=True)</pre> Example: <pre>df["status"].replace("In Progress", "Active", inplace=True)</pre>
tail()	Displays the last n rows of the DataFrame.	Syntax: <pre>dataframe.tail(n)</pre> Example: <pre>df.tail()</pre>

Numpy		
Package/Method	Description	Syntax and Code Example
Importing Numpy	Imports the Numpy library.	Syntax: <pre>import numpy as np</pre>

		Example: <pre>import numpy as np</pre>
np.array()	Creates a one or multi-dimensional array.	Syntax: <pre>array_1d = np.array([list1 values]) # 1D Array array_2d = np.array([list1 values], [list2 values]) # 2D Array</pre> Example: <pre>array_1d = np.array([1, 2, 3]) # 1D Array array_2d = np.array([[1, 2], [3, 4]]) # 2D Array</pre>
Numpy Array Attributes	- Calculates the mean of array elements - Calculates the sum of array elements - Finds the minimum value in the array - Finds the maximum value in the array - Computes dot product of two arrays	Example: <pre>np.mean(array) np.sum(array) np.min(array) np.max(array) np.dot(array_1, array_2)</pre>



© IBM Corporation. All rights reserved.